React – Fundamentals

Session – 11

1. Install Axios in your React project and use it to fetch a list of trending movies from the TMDB API (https://api.themoviedb.org/3/movie/popular) and display the first 5 movie titles in a component.

Hint: You can sign up for a free API key at TMDB or use a mock API like https://jsonplaceholder.typicode.com/posts if you want to skip authentication.

Ans.

Install Axios:

```
npm install axios
```

MovieList.jsx:

```
import { useEffect, useState } from "react";
import axios from "axios";

function MovieList() {
  const [movies, setMovies] = useState([]);

  useEffect(() => {
    axios
      .get("https://jsonplaceholder.typicode.com/posts")
      // For TMDB:
      //
.get("https://api.themoviedb.org/3/movie/popular?api_key=YOUR_API_KEY")
      .then((response) => {
        setMovies(response.data.slice(0, 5));
      })
      .catch((error) => {
        console.log(error);
      });
  }, []);

  return (
    <div>
      <h2>Top 5 Movies</h2>

      <ul>
        {movies.map((movie) => (
          <li key={movie.id}>{movie.title}</li>
        ))}
      </ul>
    </div>
```

```
  );
}

export default MovieList;
```

2. Create a React component called AddPlaylist that lets a user enter a playlist name and description, then use Axios to POST this data to https://jsonplaceholder.typicode.com/posts and show a success message when the request completes.

Ans.

AddPlaylist.jsx:

```
import { useState } from "react";
import axios from "axios";

function AddPlaylist() {
  const [playlistName, setPlaylistName] = useState("");
  const [description, setDescription] = useState("");
  const [message, setMessage] = useState("");

  const handleSubmit = async (e) => {
    e.preventDefault();

    try {
      await axios.post("https://jsonplaceholder.typicode.com/posts", {
        playlistName,
        description,
      });

      setMessage("Playlist added successfully!");

      setPlaylistName("");
      setDescription("");
    } catch (error) {
      setMessage("Failed to add playlist.");
    }
  };

  return (
    <div>
      <h2>Add Playlist</h2>

      <form onSubmit={handleSubmit}>
        <input
          type="text"
          placeholder="Playlist Name"
```

```
          value={playlistName}
          onChange={(e) => setPlaylistName(e.target.value)}
        />

        <br /><br />

        <textarea
          placeholder="Description"
          value={description}
          onChange={(e) => setDescription(e.target.value)}
        />

        <br /><br />

        <button type="submit">Add Playlist</button>
      </form>

      <p>{message}</p>
    </div>
  );
}

export default AddPlaylist;
```

3. Build a simple search bar that lets users search for restaurants by name using Axios to GET data from https://mocki.io/v1/570c5e5c-8c8b-4c1e-8c8b-4c1e8c8b4c1e (or any public mock API with a list of restaurants), filter the results as the user types, and display matching restaurant names.

Ans.

```
import { useEffect, useState } from "react";
import axios from "axios";

function RestaurantSearch() {
  const [restaurants, setRestaurants] = useState([]);
  const [search, setSearch] = useState("");

  useEffect(() => {
    axios
      // Replace with your Mocki URL if available
      .get("https://jsonplaceholder.typicode.com/users")
      .then((response) => {
        setRestaurants(response.data);
      })
      .catch((error) => {
        console.log(error);
      });
```

```
  }, []);

  const filteredRestaurants = restaurants.filter((restaurant) =>
    restaurant.name.toLowerCase().includes(search.toLowerCase())
  );

  return (
    <div>
      <h2>Restaurant Search</h2>

      <input
        type="text"
        placeholder="Search restaurant..."
        value={search}
        onChange={(e) => setSearch(e.target.value)}
      />

      <ul>
        {filteredRestaurants.map((restaurant) => (
          <li key={restaurant.id}>{restaurant.name}</li>
        ))}
      </ul>
    </div>
  );
}

export default RestaurantSearch;
```

4. Update your movie list component to handle loading and error states: show 'Loading...' while the Axios request is in progress and display an error message if the API call fails.

Hint: Use useState to track loading and error states.

Ans.

```
import { useEffect, useState } from "react";
import axios from "axios";

function MovieList() {
  const [movies, setMovies] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  useEffect(() => {
    axios
      .get("https://jsonplaceholder.typicode.com/posts")
      .then((response) => {
        setMovies(response.data.slice(0, 5));
```

```
      })
      .catch(() => {
        setError("Failed to fetch movies.");
      })
      .finally(() => {
        setLoading(false);
      });
  }, []);

  if (loading) return <h2>Loading...</h2>;

  if (error) return <h2>{error}</h2>;

  return (
    <div>
      <h2>Movie List</h2>

      <ul>
        {movies.map((movie) => (
          <li key={movie.id}>{movie.title}</li>
        ))}
      </ul>
    </div>
  );
}

export default MovieList;
```

5. Use ChatGPT or GitHub Copilot to generate an example of an Axios POST request in React, then adapt the code to submit a new comment (with fields: username and comment) to https://jsonplaceholder.typicode.com/comments and display the response below your form.

Ans.

```
import { useState } from "react";
import axios from "axios";

function CommentForm() {
  const [username, setUsername] = useState("");
  const [comment, setComment] = useState("");
  const [responseData, setResponseData] = useState(null);

  const handleSubmit = async (e) => {
    e.preventDefault();

    try {
      const response = await axios.post(
```

```
      "https://jsonplaceholder.typicode.com/comments",
      {
        username,
        comment,
      }
    );

    setResponseData(response.data);

    setUsername("");
    setComment("");
  } catch (error) {
    console.log(error);
  }
};

return (
  <div>
    <h2>Add Comment</h2>

    <form onSubmit={handleSubmit}>
      <input
        type="text"
        placeholder="Username"
        value={username}
        onChange={(e) => setUsername(e.target.value)}
      />

      <br /><br />

      <textarea
        placeholder="Comment"
        value={comment}
        onChange={(e) => setComment(e.target.value)}
      />

      <br /><br />

      <button type="submit">Submit</button>
    </form>

    {responseData && (
      <div>
        <h3>Response</h3>

        <p><strong>ID:</strong> {responseData.id}</p>
        <p><strong>Username:</strong> {responseData.username}</p>
        <p><strong>Comment:</strong> {responseData.comment}</p>
```

```
        </div>
      )}
    </div>
  );
}

export default CommentForm;
```